
Session 3 (2h) — AI for Documentation & Code Assistance

Core concepts (1h)

- **Documentation assist** — good uses: turning rough bullet notes into clean docstrings/README sections, generating a first-draft explanation of what a function does, standardizing formatting across multiple files. Weak uses: writing docs for code you haven't actually tested — the AI will confidently describe intended behavior, not verified behavior.
- **Code assist patterns:**
 - Explain this error message / this unfamiliar function
 - Review this code for bugs before I test it (catches obvious issues, doesn't replace actually running it)
 - Convert this working snippet into a different style/pattern (e.g., blocking `delay()` code → non-blocking `millis()` code)
- **Always verify, never trust blindly** — this matters especially for embedded code: a suggested fix for an interrupt handler can *look* correct and still be unsafe (e.g. forgetting `volatile`). AI output on hardware-adjacent code is a starting point for your own testing, not a final answer.
- **On-site task link** — this session is the direct rehearsal for 문서화 · 코드 작성 보조.

Hands-on practice (1h)

1. Take one `.ino` file from your Arduino prep pack (or any code you've written).
2. Ask the AI to write docstring-style comments explaining what each function does — then check each explanation against the code yourself and correct anything wrong or overstated.
3. Ask it to review the code for one specific category of bug (e.g. “check this for anything that should be `volatile` but isn't”) — verify the response against what you know about interrupts from the embedded prep pack.
4. Ask it to explain one real compiler/runtime error you've hit, and compare against your own understanding of the fix.

Checkpoint

- ☐ Generated AI-assisted documentation and personally verified it against the real code
- ☐ Used AI for a targeted code review and caught (or confirmed the absence of) a real issue
- ☐ Can explain why “verify, don't blindly trust” matters more for hardware-adjacent code